

FIAP

LEONARDO GARROTI

AQUILES FONSECA

LEANDRO DE SOUZA

CHECK POINT 01

FERRAMENTAS OPEN SOURCE DE SEGURANÇA

SÃO PAULO

2026

LEONARDO GARROTI

AQUILES FONSECA

LEANDRO DE SOUZA

CHECK POINT 01

FERRAMENTAS OPEN SOURCE DE SEGURANÇA

Relatório técnico apresentado na disciplina de **CLOUD SECURITY** da FIAP, sob orientação do professor **Fabio Pires**, como parte do Check Point 01 sobre ferramentas open source de SAST, SCA, segurança de infraestrutura como código e DAST.

SÃO PAULO

2026

SUMÁRIO

1 INTRODUÇÃO E OBJETIVO	4
2 SAST, SCA, IAC E DAST	5
3 PIPELINE, SHIFT-LEFT E SBOM	6
4 NJSSCAN: IDENTIFICAÇÃO E SITUAÇÃO DO PROJETO	7
5 NJSSCAN: FUNCIONAMENTO E COBERTURA	8
6 NJSSCAN: COMANDOS E INTEGRAÇÃO	9
7 NPM AUDIT: IDENTIFICAÇÃO E SITUAÇÃO DO PROJETO	10
8 NPM AUDIT: RESULTADO E INTERPRETAÇÃO	11
9 NPM AUDIT: COMANDOS, INTEGRAÇÃO E LIMITES	12
10 DATREE: IDENTIFICAÇÃO E SITUAÇÃO ATUAL	13
11 DATREE: FUNCIONAMENTO E EXEMPLO DE IAC	14
12 DATREE: USO E INTEGRAÇÃO PLANEJADOS	15
13 STACKHAWK: IDENTIFICAÇÃO E SITUAÇÃO ATUAL	16
14 STACKHAWK: COMO O DAST FUNCIONA	17
15 STACKHAWK: CONFIGURAÇÃO E INTEGRAÇÃO	18
16 LABORATÓRIO E AMBIENTE DOCKER	19
17 GATE E COMPARAÇÃO DOS RESULTADOS	20
18 AVALIAÇÃO CRÍTICA E CONCLUSÃO	21
REFERÊNCIAS	22
ANEXO A - USO DE IA E VALIDAÇÃO	25

1 INTRODUÇÃO E OBJETIVO

A segurança de uma aplicação depende do código, das bibliotecas e da configuração do ambiente. Algumas falhas só aparecem quando o sistema está funcionando. Por isso, este trabalho compara ferramentas que verificam partes diferentes de um projeto.

O grupo 3 é formado por Leonardo Garroti, Aquiles Fonseca e Leandro de Souza. As ferramentas indicadas para o grupo são njsscan, npm audit, Datree e StackHawk. Elas representam SAST, SCA, segurança de infraestrutura como código e DAST.

Na parte prática, usamos duas versões de uma aplicação própria em Node.js: uma com falhas intencionais e outra corrigida. A comparação mostra os alertas encontrados, as mudanças no código e a decisão do gate, que bloqueia a entrega quando encontra problemas graves.

Método usado

A pesquisa consultou a documentação e os repositórios dos projetos. O laboratório foi executado em Docker Desktop com WSL 2. O njsscan e o npm audit foram usados na prática. Datree e StackHawk foram estudados, mas não executados.

2 SAST, SCA, IAC E DAST

Cada categoria tem um objetivo. A tabela apresenta o que ela verifica e em qual condição o teste acontece.

Categoria	O que verifica	Precisa da aplicação rodando?
SAST	Código escrito pela equipe	Não
SCA	Bibliotecas e suas versões	Não
IaC	Arquivos de infraestrutura	Não
DAST	Aplicação funcionando	Sim

SAST e SCA costumam causar confusão. O SAST procura uma falha criada no próprio código. O SCA verifica se uma biblioteca usada pelo projeto possui um aviso de segurança conhecido.

No laboratório, retirar o uso de eval corrige um alerta SAST. Atualizar o lodash corrige o alerta SCA. Uma mudança não substitui a outra.

IaC ajuda a revisar arquivos como Kubernetes e Terraform antes da publicação. DAST envia requisições para a aplicação em execução. As quatro categorias se completam.

3 PIPELINE, SHIFT-LEFT E SBOM

Um pipeline executa tarefas em sequência: obter o código, instalar dependências, testar e preparar uma versão. Os testes de segurança podem entrar nessa sequência.

Etapa	Verificação adequada
Code	SAST e IaC
Build	SCA e criação da SBOM
Test	DAST
Deploy	Nova revisão da IaC

Shift-left significa trazer os testes de segurança para o início do desenvolvimento. Assim, a equipe pode corrigir uma falha antes de entregar a aplicação. O DAST completa esse processo ao testar o sistema em funcionamento.

SBOM é uma lista dos componentes usados pelo software. Ela ajuda a responder rapidamente se uma biblioteca com falha está presente. Neste trabalho, o npm gerou arquivos CycloneDX a partir dos lockfiles.

```
npm sbom --sbom-format=cyclonedx --package-lock-only
```

4 NJSSCAN: IDENTIFICAÇÃO E SITUAÇÃO DO PROJETO

O njsscan é um scanner SAST para aplicações Node.js. Seu mantenedor é Ajin Abraham. A ferramenta é escrita em Python, usa libsast e chama o Semgrep para parte da análise. A licença indicada no projeto é LGPL-3.0.

O repositório consultado foi criado em 2020. A versão usada foi a 1.0.0, publicada em 2026. Na data da pesquisa, o projeto tinha 454 estrelas e 109 forks. Esses números mostram interesse, mas não garantem manutenção contínua.

O njsscan foi escolhido porque trabalha com Node.js, gera JSON, SARIF e HTML e consegue apontar a regra, a linha e o trecho do código. Isso facilita a explicação para quem está aprendendo.

O projeto teve uma atualização recente depois de um período com poucas mudanças. Por isso, uma equipe que usar a ferramenta deve acompanhar novas versões e revisar se as regras continuam atuais.

5 NJSSCAN: FUNCIONAMENTO E COBERTURA

O njsscan procura padrões perigosos e também usa regras do Semgrep. Ele analisa a estrutura do código sem precisar iniciar a aplicação.

Na versão insegura, o relatório encontrou três problemas:

Regra	Problema	CWE
express_xss	Entrada do usuário em HTML	CWE-79
generic_os_command_exec	Entrada usada em comando	CWE-78
eval_nodejs	Entrada usada em eval	CWE-95

A leitura do código confirmou os três casos. Por isso, eles foram classificados como verdadeiros positivos. A figura apresenta o relatório HTML gerado pelo njsscan no contêiner e permite conferir as regras encontradas.

O scanner não encontra todo tipo de falha. Problemas de login, regra de negócio ou configuração em execução podem exigir outros testes.

Figura 1 - Relatório HTML do njsscan executado no contêiner

```

njsscan: v1.0.0 | Ajin Abraham | opensecurity.in
RULE ID      express_xss
OWASP-WEB    A1: Injection
CWE          CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
DESCRIPTION  Untrusted User Input in Response will result in Reflected Cross Site Scripting Vulnerability.
SEVERITY     ERROR
File         app.js
Match Position 3 - 43
FILES        Line Number(s) 8: 9
Match String  const nome = req.query.nome;
               res.send('<h1>Olá!, ' + nome + '</h1>');

RULE ID      generic_os_command_exec
OWASP-WEB    A1: Injection
CWE          CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
DESCRIPTION  User controlled data in 'child_process.exec()' can result in Remote OS Command Execution.
SEVERITY     ERROR
File         app.js
Match Position 3 - 5
FILES        Line Number(s) 12: 14
Match String  exec(req.query.cmd, function (erro, saida) {
               res.type('text').send(erro ? 'Erro' : saida);
               });

RULE ID      eval_nodejs
OWASP-WEB    A1: Injection
CWE          CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
DESCRIPTION  User controlled data in eval() or similar functions may result in Server Side Injection or Remote Code Injection
SEVERITY     ERROR
File         app.js
Match Position 21 - 41
FILES        Line Number(s) 17
Match String  const resultado = eval(req.query.expr);

```

6 NJSSCAN: COMANDOS E INTEGRAÇÃO

A instalação pode ser feita com pip. No trabalho, as versões ficam fixadas para que todos usem o mesmo ambiente.

```
pip install njsscan==1.0.0
njsscan app.js --json -o njsscan.json
njsscan app.js --sarif -o njsscan.sarif
```

JSON é simples para ler e contar alertas. SARIF pode ser enviado para plataformas de análise de código. HTML é útil para uma demonstração visual.

O arquivo de configuração permite ignorar caminhos e escolher regras. Uma regra só deve ser ignorada depois de uma revisão e de uma justificativa escrita.

No pipeline preparado, o njsscan roda antes do gate. Um alerta ERROR é tratado como nível HIGH pela regra adotada pelo grupo. Essa regra serve para o laboratório e não cria uma nota CVSS.

7 NPM AUDIT: IDENTIFICAÇÃO E SITUAÇÃO DO PROJETO

O npm audit faz parte do npm CLI. Ele é usado em projetos JavaScript e Node.js para comparar as dependências do projeto com avisos de segurança conhecidos.

O npm CLI é mantido pela equipe do npm dentro da GitHub, empresa da Microsoft. O comando audit foi adicionado em 2018. O código é JavaScript e usa a licença Artistic-2.0.

A versão usada no contêiner foi a 11.12.0. O comando lê o package-lock.json e envia informações das dependências para o serviço de auditoria do registro npm.

Como o comando já faz parte do npm, sua instalação e seu uso são simples. Porém, o resultado depende dos avisos de segurança publicados e de um lockfile que represente as dependências do projeto.

8 NPM AUDIT: RESULTADO E INTERPRETAÇÃO

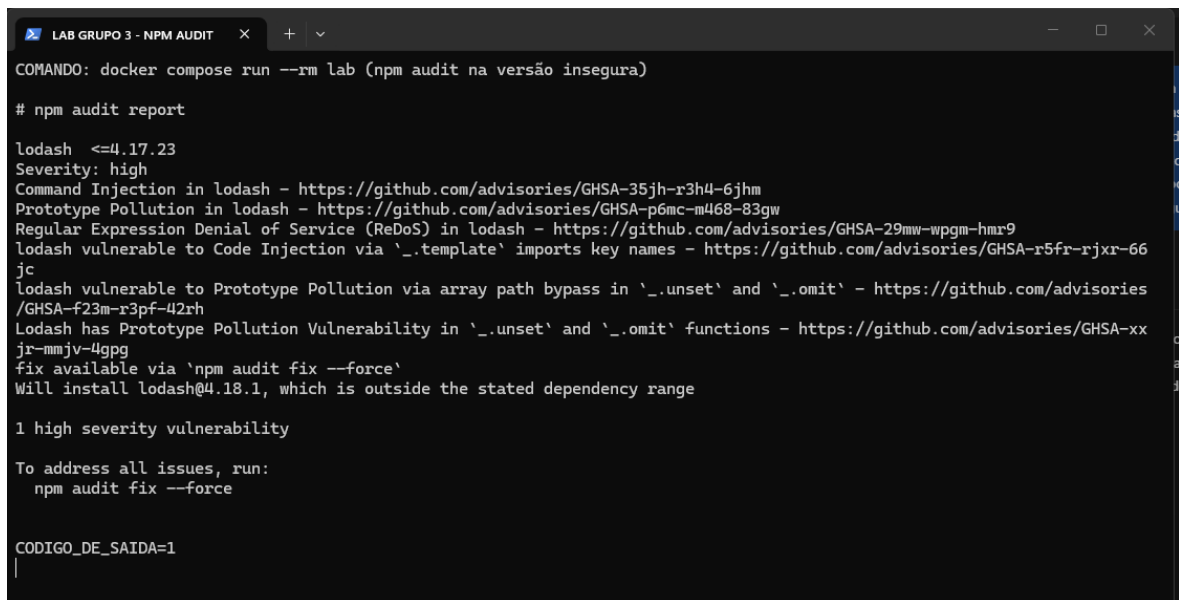
A versão insegura usa lodash 4.17.15. O npm audit encontrou um pacote com severidade HIGH. Esse pacote aparece em vários avisos, mas o resumo conta o pacote vulnerável uma vez.

A presença da versão vulnerável foi confirmada. Isso não prova que todos os avisos podem ser explorados pelas rotas do exemplo. Para responder essa pergunta, seria necessário analisar como cada função da biblioteca é usada.

No gate do laboratório, HIGH e CRITICAL bloqueiam a versão. Depois da atualização para lodash 4.18.1, o comando informou zero vulnerabilidades.

A imagem mostra a saída real do npm audit executado dentro do contêiner na versão insegura.

Figura 2 - npm audit na versão insegura



```
LAB GRUPO 3 - NPM AUDIT x + v
COMANDO: docker compose run --rm lab (npm audit na versão insegura)

# npm audit report

lodash <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Prototype Pollution in lodash - https://github.com/advisories/GHSA-p6mc-m468-83gw
Regular Expression Denial of Service (ReDoS) in lodash - https://github.com/advisories/GHSA-29mw-wpgm-hmr9
lodash vulnerable to Code Injection via `_.template` imports key names - https://github.com/advisories/GHSA-r5fr-rjxr-66jc
lodash vulnerable to Prototype Pollution via array path bypass in `_.unset` and `_.omit` - https://github.com/advisories/GHSA-f23m-r3pf-42rh
lodash has Prototype Pollution Vulnerability in `_.unset` and `_.omit` functions - https://github.com/advisories/GHSA-xxjr-mmjv-4gpg
fix available via `npm audit fix --force`
Will install lodash@4.18.1, which is outside the stated dependency range

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force

CODIGO_DE_SAIDA=1
```

9 NPM AUDIT: COMANDOS, INTEGRAÇÃO E LIMITES

Os comandos principais são curtos:

```
npm audit
npm audit --audit-level=high
npm audit --json
npm audit fix
```

`--audit-level=high` define o nível usado para o código de saída. `--json` cria um resultado que pode ser lido pelo gate. `npm audit fix` tenta atualizar pacotes dentro dos limites permitidos pelo projeto.

O uso de `--force` precisa de cuidado, pois pode instalar uma versão fora da faixa declarada. A equipe deve executar testes depois da atualização.

O npm audit pode entrar no CI logo depois da instalação das dependências. Ele também pode ser usado antes do commit, mas consultas frequentes dependem da rede e do serviço do registro.

Um resultado com zero vulnerabilidades significa apenas que nenhum aviso conhecido foi encontrado nas dependências analisadas naquele momento.

10 DATREE: IDENTIFICAÇÃO E SITUAÇÃO ATUAL

Datree foi uma ferramenta de segurança para arquivos Kubernetes. O CLI era escrito em Go e distribuído com licença Apache-2.0. Ele avaliava arquivos YAML antes de chegarem ao cluster.

O projeto foi arquivado no GitHub em outubro de 2024. A própria empresa informou o fim da manutenção em 2023. A última versão encontrada foi a 1.9.19.

A falta de manutenção limita sua adoção. O Datree ainda serve para estudar políticas de Kubernetes, mas um projeto novo deve considerar uma ferramenta que continue recebendo atualizações.

O enunciado atribui Datree ao grupo, por isso o nome foi mantido na pesquisa. Para um uso real atual, seria melhor comparar alternativas mantidas, como Checkov, Trivy ou Kubescape.

11 DATREE: FUNCIONAMENTO E EXEMPLO DE IAC

O Datree lê um arquivo Kubernetes, verifica se o YAML é válido e aplica regras de política. Uma regra pode avisar sobre privilégio alto, ausência de limites de recursos ou uso inseguro de imagem.

No exemplo inseguro do trabalho, o contêiner usa privilégio elevado e não define limites. Na versão corrigida, o privilégio é retirado, o sistema de arquivos fica somente para leitura e os recursos recebem limites.

```
datree test iac/pod-inseguro.yaml --schema-version 1.20.0
```

A análise acontece antes do deploy. Isso reduz a chance de uma configuração ruim chegar ao cluster.

A ferramenta não verifica o comportamento da aplicação, a segurança da imagem ou as permissões reais do ambiente. Esses pontos precisam de outras verificações.

12 DATREE: USO E INTEGRAÇÃO PLANEJADOS

O CLI podia ser instalado por script, pacote ou contêiner. Também podia rodar em GitHub Actions, GitLab CI e outros sistemas de CI.

```
datree test iac/pod-inseguro.yaml --output json
```

Uma equipe podia criar políticas próprias e definir quais regras bloqueariam o pipeline. O modo offline permitia usar regras guardadas localmente.

Neste trabalho, o Datree não foi executado porque o projeto está arquivado. Foram mantidos apenas os arquivos de exemplo e a explicação do funcionamento.

Para adotar uma alternativa, o grupo deveria repetir o mesmo processo do laboratório: fixar versão, executar contra um exemplo conhecido, guardar o relatório e definir um gate claro.

13 STACKHAWK: IDENTIFICAÇÃO E SITUAÇÃO ATUAL

StackHawk é uma plataforma de testes DAST. Seu scanner é chamado HawkScan e utiliza uma base ligada ao OWASP ZAP. O uso comum acontece por contêiner.

A empresa StackHawk mantém a plataforma e a documentação. O serviço possui partes comerciais. Isso precisa ser explicado porque o trabalho pede ferramentas open source. A base do scanner tem origem aberta, mas a experiência completa depende da plataforma.

A versão 6.0.0 do HawkScan foi anunciada em 2026. O produto continua ativo. Mesmo assim, a equipe deve conferir plano, limites e necessidade de conta antes de usar em um projeto.

O StackHawk foi mantido na pesquisa por estar na lista do grupo. Ele não foi usado no laboratório obrigatório.

14 STACKHAWK: COMO O DAST FUNCIONA

O DAST testa a aplicação funcionando. Ele descobre rotas, envia entradas de teste e observa respostas. Isso permite encontrar problemas que dependem do servidor e da configuração do ambiente.

O fluxo básico é:

Passo	Ação
1	Iniciar a aplicação de teste
2	Definir o endereço permitido
3	Executar o scanner
4	Revisar os alertas
5	Corrigir e testar novamente

A autenticação exige configuração extra quando as rotas são protegidas. Sem isso, o scanner pode enxergar apenas a tela de login.

DAST pode gerar falso positivo e também deixar rotas sem teste. Ele deve ser executado somente contra um alvo próprio ou autorizado.

15 STACKHAWK: CONFIGURAÇÃO E INTEGRAÇÃO

A configuração usa um arquivo YAML com o endereço da aplicação, o identificador do projeto e opções de ambiente. Segredos devem ficar nas variáveis protegidas do CI.

```
docker run --rm -v "$PWD:/hawk" stackhawk/hawkscan:latest
```

Em uma equipe, é melhor fixar a versão da imagem em vez de usar `latest`. Isso evita que uma atualização mude o resultado sem aviso.

O scanner pode rodar depois que o ambiente de teste estiver disponível. O relatório precisa ser guardado como artefato. Antes de bloquear o build, a equipe deve revisar quais níveis são confiáveis para o projeto.

StackHawk não substitui SAST ou SCA. Ele observa o sistema por fora e não conhece todo o caminho interno do código.

16 LABORATÓRIO E AMBIENTE DOCKER

O laboratório usa uma aplicação Node.js criada para a atividade. Há um alvo inseguro e outro corrigido. Os dois ficam na mesma imagem para facilitar a comparação.

O Dockerfile instala Node.js, Python, njsscan, Sengrep e npm. O Docker Compose executa o script de análise e grava os relatórios. A imagem criada recebeu o nome `cp01-grupo3:1.0`.

A construção e as duas execuções foram realizadas no Docker Desktop com WSL 2. A tela do Docker Desktop confirma a imagem local. A tela do terminal registra as versões e o identificador da imagem.

Esse ambiente evita que cada colega precise instalar todas as ferramentas diretamente no Windows.

Figura 3 - Imagem cp01-grupo3 no Docker Desktop

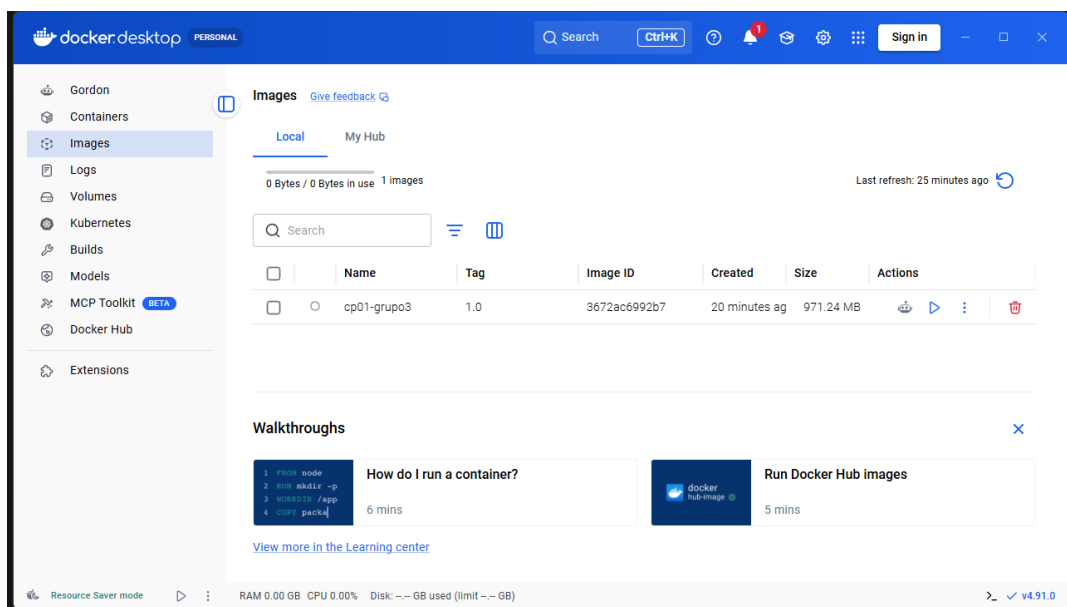


Figura 4 - Ambiente Docker e WSL 2

```
LAB GRUPO 3 - DOCKER
COMANDO: docker compose build --pull lab

Docker version 29.8.0, build 88096ef
Docker Compose version v5.5.1
WSL: Ubuntu, versão 2

Imagem: cp01-grupo3:1.0
ID: sha256:3672ac6992b77b006867348d9b05d2015cb33269a723ac86014ce11688ef4f25
Criada em: 2026-09-16T18:55:28.743786323Z
Tamanho: 971245040 bytes

RESULTADO: imagem criada com sucesso.
```

17 GATE E COMPARAÇÃO DOS RESULTADOS

O gate lê os relatórios do njsscan e do npm audit. A política usada foi:

Resultado	Decisão
SAST ERROR	Bloqueia como HIGH
SCA HIGH ou CRITICAL	Bloqueia
Sem alertas nesses níveis	Aprova

Na versão insegura, três alertas SAST e um pacote HIGH produziram código de saída 1. O status foi BLOQUEADO.

Na versão corrigida, os dois scanners terminaram com código 0. O status foi APROVADO.

O print abaixo registra o bloqueio da versão insegura. Na página seguinte, os prints mostram a aprovação após as correções. Esses resultados comprovam os testes locais no Docker.

Figura 5 - Gate bloqueando a versão insegura

```

LAB GRUPO 3 - TESTE BLOQU  X  +  v
COMANDO: docker compose run --rm lab inseguro

SAIDA REAL DO CONTAINER:
{
  "alvo": "inseguro",
  "status": "BLOQUEADO",
  "exit_code": 1,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 3,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 1,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  }
}

njsscan-json: 4.809s, codigo 1
njsscan-sarif: 4.731s, codigo 1
npm-audit: 0.731s, codigo 1

CODIGO_DE_SAIDA=1
|

```

18 AVALIAÇÃO CRÍTICA E CONCLUSÃO

O laboratório mostrou por que SAST e SCA são diferentes. O njsscan encontrou falhas escritas no app.js. O npm audit encontrou uma versão vulnerável do lodash.

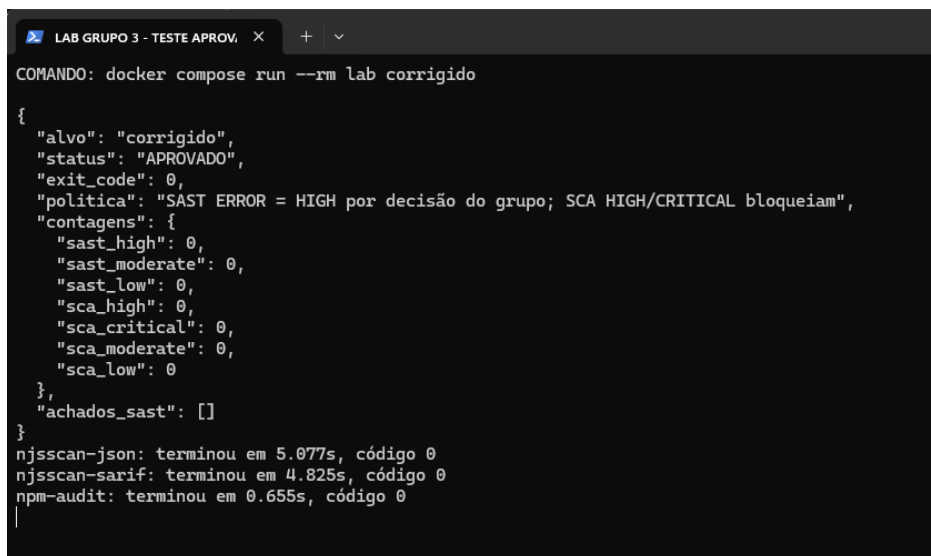
As correções também foram diferentes: retirar operações perigosas no código e atualizar a dependência. Depois disso, o gate aprovou a nova versão e o npm audit mostrou zero vulnerabilidades.

O njsscan depende das regras disponíveis; o npm audit, dos avisos conhecidos. Datree está arquivado e StackHawk possui partes comerciais.

Para um projeto pequeno, a sequência sugerida é SAST e SCA desde o começo, análise de IaC antes do deploy e DAST em um ambiente de teste autorizado.

A aprovação confirma a correção dos alertas analisados. Outros testes e a revisão do código continuam necessários.

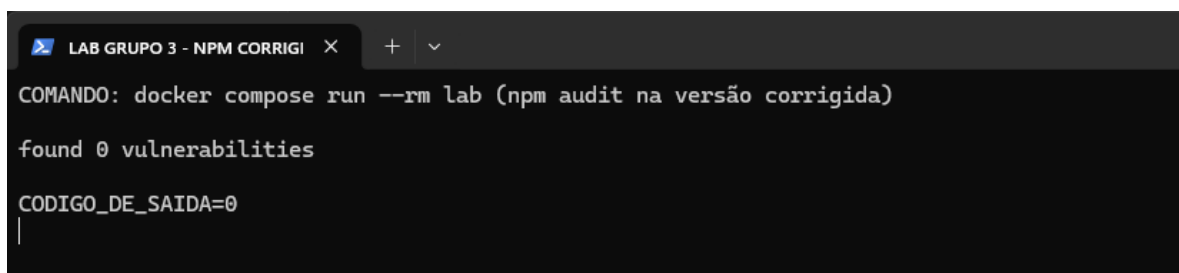
Figura 6 - Gate aprovando a versão corrigida



```
LAB GRUPO 3 - TESTE APROV. x + v
COMANDO: docker compose run --rm lab corrigido

{
  "alvo": "corrigido",
  "status": "APROVADO",
  "exit_code": 0,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 0,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 0,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  },
  "achados_sast": []
}
njsscan-json: terminou em 5.077s, código 0
njsscan-sarif: terminou em 4.825s, código 0
npm-audit: terminou em 0.655s, código 0
|
```

Figura 7 - npm audit corrigido: zero vulnerabilidades



```
LAB GRUPO 3 - NPM CORRIGI x + v
COMANDO: docker compose run --rm lab (npm audit na versão corrigida)

found 0 vulnerabilities

CODIGO_DE_SAIDA=0
|
```

REFERÊNCIAS

ABRAHAM, Ajin. **njsscan**. 2026. Disponível em: <https://github.com/ajinabraham/njsscan>.

ABRAHAM, Ajin. **njsscan: setup.py**. 2026. Disponível em: <https://github.com/ajinabraham/njsscan/blob/master/setup.py>.

ABRAHAM, Ajin. **njsscan: interface de linha de comando**. 2026. Disponível em: https://github.com/ajinabraham/njsscan/blob/master/njsscan/__main__.py.

ABRAHAM, Ajin. **libsast: chamada do motor Semgrep**. s.d.. Disponível em: https://github.com/ajinabraham/libsast/blob/master/libsast/core_sgrep/helpers.py.

DATREE. **Datree: aviso de descontinuação**. 2023. Disponível em: <https://github.com/datreeio/datree>.

DATREE. **Release 1.9.19**. 2023. Disponível em: <https://github.com/datreeio/datree/releases/tag/1.9.19>.

DATREE. **Offline mode**. s.d.. Disponível em: <https://hub.datree.io/cli/offline-mode>.

DATREE. **CLI arguments**. s.d.. Disponível em: <https://hub.datree.io/cli/cli-arguments>.

GITHUB. **SARIF support for code scanning**. s.d.. Disponível em: <https://docs.github.com/en/code-security/reference/code-scanning/sarif-files/sarif-support>.

GITHUB ADVISORY DATABASE. **Prototype Pollution in Iodash: CVE-2020-8203**. 2020. Disponível em: <https://github.com/advisories/GHSA-p6mc-m468-83gw>.

REFERÊNCIAS (CONTINUAÇÃO)

GITHUB ADVISORY DATABASE. **Lodash vulnerable to Code Injection: CVE-2026-4800**. 2026. Disponível em: <https://github.com/advisories/GHSA-r5fr-rjxr-66jc>.

MITRE. **CWE-78: OS Command Injection**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/78.html>.

MITRE. **CWE-79: Cross-site Scripting**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/79.html>.

MITRE. **CWE-95: Eval Injection**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/95.html>.

NIST. **Secure Software Development Framework (SSDF), SP 800-218**. 2022. Disponível em: <https://csrc.nist.gov/pubs/sp/800/218/final>.

NPM. **npm-audit**. s.d.. Disponível em: <https://docs.npmjs.com/cli/v11/commands/npm-audit/>.

NPM. **npm-sbom**. s.d.. Disponível em: <https://docs.npmjs.com/cli/v11/commands/npm-sbom/>.

NPM. **Changelog: npm v6**. 2018. Disponível em: <https://docs.npmjs.com/cli/v6/using-npm/changelog/>.

NPM. **Licença da aplicação npm**. s.d.. Disponível em: <https://github.com/npm/cli/blob/latest/LICENSE>.

NPM. **Registro do pacote npm**. 2026. Disponível em: <https://registry.npmjs.org/npm>.

REFERÊNCIAS (CONTINUAÇÃO)

OWASP. **Top 10:2025**. 2025. Disponível em: https://top10.owasp.org/2025/0x00_2025-Introduction/.

PYPI. **njsscan 1.0.0**. 2026. Disponível em: <https://pypi.org/project/njsscan/1.0.0/>.

STACKHAWK. **HawkScan**. s.d.. Disponível em: <https://docs.stackhawk.com/hawkscan/>.

STACKHAWK. **StackHawk CLI**. s.d.. Disponível em: <https://docs.stackhawk.com/stackhawk-cli/>.

STACKHAWK. **HawkScan Configuration**. s.d.. Disponível em: <https://docs.stackhawk.com/hawkscan/configuration/>.

STACKHAWK. **HawkScan 6.0.0 Release Notes**. 2026. Disponível em: <https://docs.stackhawk.com/hawkscan/6-0-0/>.

STACKHAWK. **Changelog**. s.d.. Disponível em: <https://docs.stackhawk.com/changelog/>.

STACKHAWK. **StackHawk Announces HawkScan Test Engine**. 2024. Disponível em: <https://www.stackhawk.com/blog/stackhawk-announces-hawkscan-test-engine/>.

STACKHAWK. **Announcing Series B Funding**. 2022. Disponível em: <https://www.stackhawk.com/blog/announcing-stackhawks-207-million-in-series-b-funding-to-drive-developer-first-security/>.

ANEXO A. USO DE IA E VALIDAÇÃO

Usamos IA como apoio na escrita, na revisão e na organização do trabalho, dos slides e do laboratório. Os resultados apresentados foram obtidos com a execução das ferramentas e conferidos com o código e a documentação. Os integrantes devem revisar o material e compreender os testes antes da entrega e da apresentação.

Conferência dos resultados

O laboratório foi executado no Docker Desktop com WSL 2. A versão insegura foi bloqueada com código 1; a versão corrigida foi aprovada com código 0. Os prints registram essas execuções locais. Datree e StackHawk foram pesquisados, mas não executados.

Preparação para a apresentação

Os três integrantes devem compreender os alertas, as correções e os comandos do laboratório. As contribuições de cada pessoa precisam ser registradas de verdade no repositório. A atividade individual com o laboratório de outro grupo também deve ser feita por cada integrante.

Dados medidos

Teste	Inseguro (s)	Corrigido (s)
njsscan-json	4.809	5.077
njsscan-sarif	4.731	4.825
npm-audit	0.731	0.655